PROJECT REPORT

IoT-Based Vehicle Accident Detection and Alert System

A simulation-first system integrating vehicle sensors, MQTT, backend incident processing, ML evaluation and a live dashboard

STUDENT	PROJECT
MUKESH S	IoT-Based Vehicle Accident Detection and Alert System
Register No.	711524BCS095
Department / Institution	_____
Project Guide	_____
Academic Year	2026–2027

Submitted in partial fulfilment of the requirements for the award of the degree

Report prepared: 28 October 2026

CERTIFICATE

This is to certify that the project report entitled “IoT-Based Vehicle Accident Detection and Alert System” is prepared by MUKESH S (Register No. 711524BCS095) as a project study. The report describes the repository implementation and simulation status available on 28 October 2026.

Project Guide	Head of Department	External Examiner
Signature: _____	Signature: _____	Signature: _____
Name: _____	Name: _____	Date: _____

DECLARATION

I declare that this report is prepared for the stated project and that implementation status, test results, assumptions and limitations are identified as accurately as possible from the project source code and existing evaluation artifacts. The simulation is not a certified automotive safety device.

Student signature: _____ Date: _____

ACKNOWLEDGEMENT

I thank my project guide, department and institution for their guidance and support. I also acknowledge the open-source software and simulation tools used in this project, including Wokwi, Eclipse Mosquitto, FastAPI, TensorFlow-related tooling, MongoDB, Node.js and React. The report distinguishes between software that is implemented, offline evaluation artifacts, and live integrations that still require configuration.

ABSTRACT

Road accidents require timely recognition and reliable delivery of a vehicle's location to responders. This project presents a simulation-first IoT accident detection and alert system built around an ESP32 and MPU6050 accelerometer/gyroscope. Sensor samples are represented as telemetry and, when MQTT connectivity is available, pass through a Mosquitto broker to a Node.js backend. The backend maintains per-device sliding windows, derives impact and motion features, requests classification from a FastAPI service, records incident state in MongoDB, and streams status and incident updates to a React dashboard using Socket.IO. A buzzer and driver cancel button model local alert and false-alarm handling. GPS coordinates in the Wokwi circuit are fixed fallback values; a GPS receiver is not simulated.

The repository includes a 20,500-row simulator dataset and saved evaluation artifacts comparing a model-labelled classifier and an autoencoder baseline. The saved comparison reports 99.35% accuracy and 98.69% recall for the DNN-labelled result, while a three-times-noise stress result has only 28.15% accuracy. These numbers are simulator/report artifacts, not evidence of field performance. Code inspection shows that the current prediction label is computed by physical threshold rules rather than a learned DNN forward pass; the project must address this discrepancy before claiming deployed DNN inference. MOMENT evaluation was not completed. Local health endpoints were previously verified, but a live Wokwi-to-local-broker run requires Wokwi Private IoT Gateway, which was unavailable during integration verification.

The project demonstrates the end-to-end architecture and provides a useful educational prototype. Real-world use requires a trained and independently validated model, live GPS, secure broker credentials, field testing, fault handling and safety review.

KEYWORDS

Internet of Things, ESP32, MPU6050, MQTT, accident detection, sliding window, anomaly detection, DNN, Wokwi, Socket.IO.

CONTENTS

- 1. Introduction and objectives
- 2. Requirements and related technical concepts
- 3. System architecture and design
- 4. Detection and incident methodology
- 5. Implementation
- 6. Evaluation and testing
- 7. Limitations, safety and future work
- 8. Conclusion and references
- Appendix A. Run and demonstration procedure
- Appendix B. MQTT message contract

1. INTRODUCTION AND OBJECTIVES

1.1 Background

After a road collision, a vehicle may be unable to report the event and its location. An embedded device can observe acceleration and angular velocity, identify an abnormal event, and send an alert through a connected service. The detection decision must distinguish a collision from non-accident events such as potholes, hard braking and sharp turns. The system also needs an operator or driver cancellation path to reduce false alarms.

1.2 Problem statement

The project addresses the design of an affordable prototype that (i) senses impact-like motion, (ii) evaluates the event using a time window rather than a single sample, (iii) attaches a location field, and (iv) raises a reviewable alert through a dashboard. A simulation environment is used to develop and demonstrate the software flow without requiring a physical vehicle.

1.3 Objectives

- Represent ESP32, MPU6050, buzzer and a cancel input in the Wokwi circuit.
- Publish status, telemetry and event records using a consistent MQTT topic contract.
- Compute window-based motion features and classify candidate events in the backend/ML service.
- Track pending, cancelled and confirmed incidents and display device/incident activity in the dashboard.
- Compare the repository's classifier-labelled and autoencoder evaluation artifacts; report the limitations honestly.

1.4 Scope

This is a software and hardware simulation prototype. It demonstrates data contracts, service integration, user-interface flow and a device alert model. It does not establish a safe crash detector, guarantee communications coverage, contact emergency services, or validate a production ML model. The real GPS hardware path and live Wokwi-to-local MQTT connection remain incomplete unless corresponding equipment and gateway access are configured.

1.5 Project particulars

Item	Project brief / implementation
Topic	IoT-Based Vehicle Accident Detection and Alert System
Board and sensors	ESP32; MPU6050 accelerometer/gyroscope; GPS is a planned real sensor and fallback in Wokwi
Alert output	Buzzer, hardware cancel button, dashboard incident state
Communication	MQTT from device to backend; Socket.IO from backend to web client
Planned data / methods	Vehicle sensor data; DNN classification; MOMENT and autoencoder baselines
Budget from supplied brief	Indicative BOM ₹2,550; recommended student allocation up to ₹5,000 within stated ₹5,000–₹10,000 target. Confirm actual vendor prices.

2. REQUIREMENTS AND TECHNICAL CONCEPTS

2.1 Functional requirements

ID	Requirement	Implemented evidence
FR-01	Acquire tri-axial acceleration and gyro readings.	Wokwi MPU6050 I2C firmware and Node simulator sensors.
FR-02	Send device status and telemetry.	MQTT status/telemetry topics; backend subscriber.
FR-03	Screen a rolling sensor window for an impact candidate.	Per-device ring buffer; statistical feature extractor; screening gates.
FR-04	Create and manage an incident.	Pending confirmation, debounce, countdown, cancellation and confirmation service.
FR-05	Show fleet readings and alerts.	React Fleet Monitor and Socket.IO event streams.
FR-06	Allow false-alarm cancellation.	Wokwi GPIO 4 button and backend/device command paths.

2.2 Non-functional requirements

- Timeliness: telemetry is sampled at 50 Hz in the Wokwi sketch; frontend telemetry is throttled by the backend.
- Modularity: MQTT ingestion, incident logic, ML service and dashboard are separate components.
- Traceability: device IDs, timestamps, sequence values, event types and sensor snapshots support diagnosis.
- Security: the local development broker is anonymous; it must not be exposed publicly. Production authentication and TLS are required.
- Safety: a software alert is advisory only; thresholds and model outputs require vehicle-grade validation.

2.3 Technical concepts

MQTT is a lightweight publish/subscribe protocol. A topic groups data by vehicle and type (for example, vehicle/{id}/telemetry). A sliding window retains recent samples so features such as peak acceleration, jerk, speed drop and gyro magnitude can be evaluated together. The backend screening gate avoids an ML request for quiet windows; accepted candidates enter an incident state machine with a cancellation countdown. Socket.IO pushes updates to connected dashboard clients.

The repository supports a simulator-generated/exported data path and a synthetic-data fallback. This report does not claim that a public vehicle dataset has been imported: the committed runtime dataset contains simulator samples. Further work should document dataset provenance, licensing, sensor calibration and a subject-/run-independent test protocol.

3. SYSTEM ARCHITECTURE AND DESIGN

The system is organized as a device-to-dashboard pipeline. The Wokwi firmware models hardware behavior; the virtual fleet simulator provides additional generated scenarios; both can publish to the same MQTT topic family. The backend ingests telemetry and events, builds per-device windows, evaluates a candidate, records incident data, and emits real-time updates.

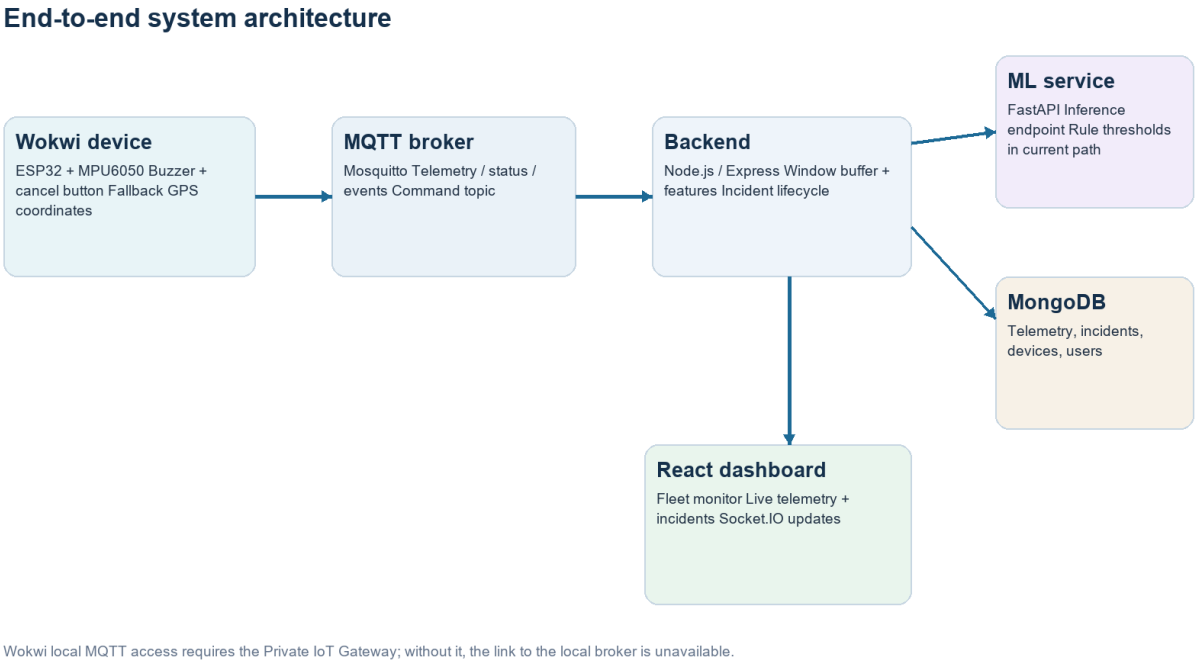


Figure 1. Software and messaging architecture. Local Wokwi-to-broker traffic requires the Private IoT Gateway.

Component	Technology / role
Device simulation	ESP32 DevKit + MPU6050 in Wokwi; separate Node.js virtual vehicle simulator.
Message transport	Eclipse Mosquitto broker; telemetry, status and event topics.
Backend	Node.js / Express, MQTT subscriber, sliding-window service, incident service and REST API.
ML service	Python / FastAPI; /predict, /train, /health and /metrics endpoints.
Storage	MongoDB for users, devices, telemetry and incidents.
Frontend	React + Vite; Fleet Monitor, incident views and Socket.IO live updates.

3.1 Data flow

1. The device samples the MPU6050 and constructs a JSON telemetry message with device ID, epoch-millisecond timestamp, sequence, accelerometer, gyro and GPS fields.
2. The MQTT broker delivers status, telemetry and event messages to the backend subscriber.
3. Telemetry is normalized into a per-device buffer; a short feature window is screened for high acceleration, speed loss or rotation.
4. Candidate features are sent to the ML service. The result is checked against a confidence threshold before the backend creates an incident.
5. Incident and device updates are persisted and broadcast to the React client; operator actions may publish a command back to the device.

4. DETECTION AND INCIDENT METHODOLOGY

4.1 Sensor inputs and feature extraction

The firmware reads three accelerometer and three gyroscope axes from the MPU6050. In Wokwi, the sensor control supplies simulated values. The shared feature extractor computes a 14-element vector over a sample window.

Feature group	Features
Acceleration	Peak total acceleration, mean magnitude, RMS, standard deviation, mean Ax/Ay/Az
Change over time	Maximum jerk in g/s
Vehicle speed	Initial speed, final speed, speed delta, speed-drop ratio
Rotation	Maximum and mean gyro magnitude (degrees/s)

Acceleration magnitude is calculated as $\sqrt{(ax^2 + ay^2 + az^2)}$. Jerk approximates the change in acceleration magnitude divided by elapsed time. The server and Python implementation state that they share the same 14-feature definition.

4.2 Screening and classification

The backend uses a two-second default window and a 50% overlap setting in its sliding-window service. A candidate is screened when peak G, speed-drop ratio or gyro magnitude exceeds a configured value. The ML client requests the FastAPI /predict endpoint and retries once; if the service cannot be reached, a rule engine is used as fallback.

Implementation qualification: although files and saved reports use the DNN name, current model.py computes the final predict_from_features label through _rule_based_fallback thresholds. The training routine also derives its reported DNN-labelled predictions from these rules. The autoencoder baseline uses a fixed projection/reconstruction scheme and a threshold from normal-window reconstruction errors; it is not shown as a gradient-trained neural network. Therefore the stored metrics should not be presented as independently verified learned-DNN performance.

4.3 Incident state and alerting

- A qualifying accident candidate creates a PENDING_CONFIRMATION incident unless an active incident or debounce period prevents duplicates.
- The default cancellation countdown is 15 seconds. The driver cancel button can mark the pending incident as a false alarm and silence the buzzer.
- If the countdown expires, the incident is confirmed and alert dispatch is attempted according to configured channels.
- Events and status are delivered to the dashboard. External SMS/email delivery depends on provider credentials and configuration.

4.4 Location handling

The Wokwi firmware currently places fixed fallback coordinates 10.9982, 76.9664 in telemetry. It does not emulate GPS satellite acquisition, accuracy, loss of fix or movement. The application can display a location field, but Wokwi location output must not be represented as a live GPS reading.

5. IMPLEMENTATION

5.1 Wokwi hardware model

The Wokwi circuit contains an ESP32 DevKit C, an MPU6050, a buzzer and a red CANCEL pushbutton. The firmware configures Wi-Fi, waits for NTP time, publishes MQTT messages where connectivity allows, processes inbound commands, samples the sensor and maintains impact/alert state.

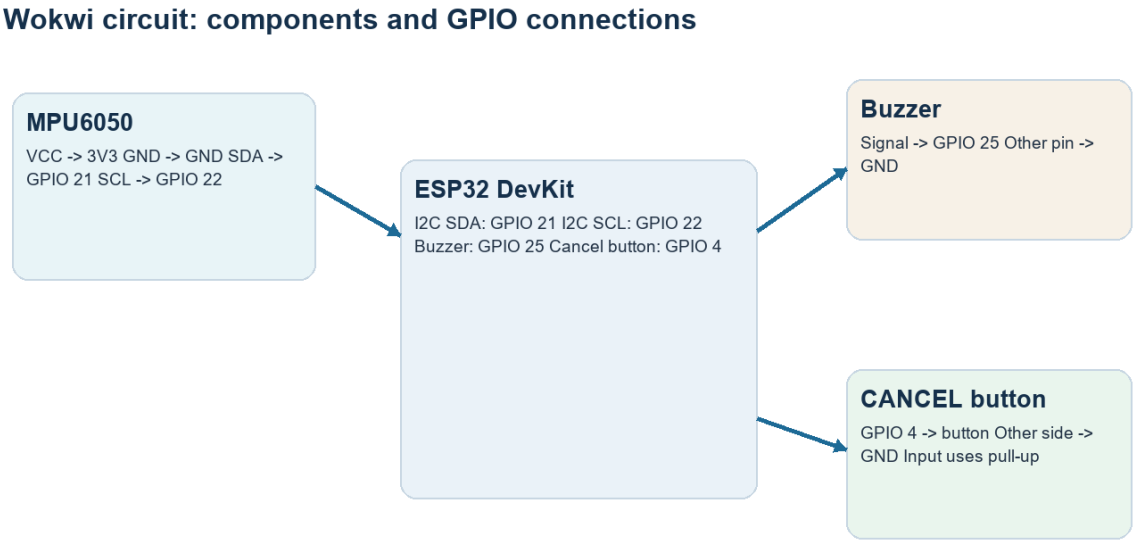


Figure 2. Circuit components and GPIO/I2C wiring derived from wokwi/diagram.json.

5.2 Software modules

Module	Responsibilities
wokwi/sketch.ino	Sensor read, local impact state, buzzer, cancel button, MQTT status/telemetry/events and commands.
server/src/mqtt/subscriber.js	Subscribe to vehicle topics; normalize telemetry and route status/event updates.
server/src/services/slidingWindowService.js	Per-device ring buffers, evaluation cadence, screening and incident candidates.
ml-service/app/features.py	Canonical statistical and kinematic window feature extraction.
ml-service/app/model.py	Model-labelled predictions, threshold classifier, autoencoder baseline and saved metrics.
server/src/services/incidentService.js	Incident persistence, debounce, countdown, cancellation and confirmation.
client/src/pages/FleetPage.jsx	Online device list, live acceleration, speed, route and location display.

5.3 MQTT topic contract

Topic	Direction	Purpose
vehicle/{id}/status	Device → backend	Online/offline status, plate, route and timestamp; retained status / Last Will.
vehicle/{id}/telemetry	Device → backend	50 Hz sample with accel, gyro, GPS fields, buzzer and timestamp.
vehicle/{id}/event	Device → backend	Impact detected, alert cancelled or alert confirmed.
vehicle/{id}/command	Backend → device	Buzzer ON/OFF or CANCEL/RESET commands.

6. EVALUATION AND TESTING

6.1 Evaluation data and results

The runtime simulator dataset file contains 20,500 rows (2,205 accident-labelled and 18,295 non-accident-labelled rows). The stored model comparison artifact reports results on a 3,098-sample evaluation split. These are repository-generated or repository-recorded simulation results, not results from real vehicle crashes.

Artifact	Accuracy	Precision	Recall	F1	Interpretation
DNN-labelled result	99.35%	94.95%	98.69%	96.78%	Reported in model_comparison.json; prediction code is threshold-rule based.
Autoencoder-labelled baseline	96.97%	83.92%	85.57%	84.74%	Stored baseline; fixed projection reconstruction.

Stored DNN-labelled confusion matrix: TN=2,777, FP=16, FN=4, TP=301 (evaluation artifact). Stored autoencoder matrix: TN=2,743, FP=50, FN=44, TP=261. Metrics are rounded and should be re-produced with a documented, independent test set before use in a formal performance claim.

Domain-shift artifact	Accuracy	Precision	Recall	F1	Samples
DNN-labelled	96.73%	80.69%	97.64%	88.36%	3000
Rule engine	98.27%	100.00%	86.35%	92.68%	3000

6.2 Stress test and baseline status

The saved 3× noise stress artifact reports accuracy 28.15%, precision 12.05%, recall 100.00% and F1 21.51%. The high recall is accompanied by many false positives (2,226 in the saved confusion matrix). This indicates poor robustness to the tested perturbation and is not acceptable evidence of production readiness.

MOMENT was not evaluated. The evaluation record reports NOT_EVALUATED because the PyTorch runtime could not load torch.dll under the current Windows application-control policy. Any report of a MOMENT score would therefore be unsupported.

6.3 Integration checks

Check	Observed status / qualification
Local service health	Backend, ML service, simulator and frontend health endpoints previously returned HTTP 200.
MQTT broker	Local Mosquitto port 1883 was listening when checked; Wokwi private gateway port 9011 was not listening.
ML inference	A prior normal-window /predict request returned NON_ACCIDENT with confidence 0.04.
Local MQTT → backend → Socket.IO	Previously verified with a connected socket client for online status and telemetry.
Wokwi compile and basic simulation	Prior browser compile/boot reported READY and TIME_SYNCHRONIZED; hardware simulation was run.
Live Wokwi → local broker → dashboard incident	Not verified: Private IoT Gateway was unavailable; public gateway cannot reach local services.

7. LIMITATIONS, SAFETY AND FUTURE WORK

7.1 Current limitations

- No physical vehicle or road testing is represented; simulated accelerometer controls are not calibrated crash forces.
- Wokwi GPS values are fixed fallback coordinates. No actual GPS module or live fix is shown.
- Private Gateway access is required for Wokwi to connect to the local MQTT broker; this was not enabled during the integration verification.
- The anonymous development MQTT broker is suitable only for local development and must not be exposed to the internet.
- Current inference code returns a threshold-rule classification despite DNN naming in interfaces/artifacts; the DNN claim requires a genuine trained model and a verified inference path.
- Autoencoder baseline is a simplified fixed projection, not a fully optimized neural autoencoder; MOMENT was not evaluated.
- The extreme-noise evaluation produces a high false-positive burden; domain-shift metrics show a precision/recall trade-off.
- Email/SMS alerts depend on external provider configuration; successful in-app incident state does not establish emergency-service delivery.

7.2 Safety and responsible use

This prototype is not an emergency-response guarantee, a medical or safety-certified device, or a replacement for a vehicle's airbag/telematics system. False negatives could delay assistance; false positives can create unnecessary dispatches. A real deployment requires rigorous sensor placement and calibration, secure identities and transport, availability engineering, human confirmation policy, documented incident response, privacy controls for location history, and independent testing against representative events.

7.3 Recommended future work

6. Replace the rule-only prediction label with a genuine trained and persisted DNN, load it in /predict, and write unit tests proving that inference uses model weights.
7. Acquire and cite a legally usable public vehicle sensor dataset; preserve provenance and split by vehicle/driver/run to prevent leakage.
8. Train and calibrate a proper autoencoder on normal-only windows; evaluate threshold selection on an independent validation set.
9. Evaluate MOMENT only in a compatible environment and report its compute requirements, baseline and failure cases.
10. Add a simulated or physical GPS receiver with no-fix, accuracy and stale-location handling.
11. Enable Wokwi Private IoT Gateway and perform the full Wokwi → MQTT → backend → ML → MongoDB → frontend incident/cancel test.
12. Harden the broker with authentication, per-device ACLs, TLS, secret management and network isolation.
13. Expand tests to include latency, packet loss, reconnect behavior, false alarms, cancellation races and field-representative sensor noise.

8. CONCLUSION

The project assembles a coherent simulation-first IoT architecture for accident-event monitoring. Its software surfaces cover ESP32 telemetry, MQTT ingestion, rolling feature windows, candidate classification, incident state transitions, persistent data and a real-time dashboard. The Wokwi circuit models an MPU6050, buzzer and cancellation button, while the virtual fleet simulator supports additional generated non-accident and accident scenarios.

The deliverable is an educational prototype rather than a complete operational accident-alert system. The live Wokwi-to-local-broker path remains gated by the Wokwi Private IoT Gateway, and GPS is a fixed fallback in Wokwi. Most importantly, the current classifier implementation does not substantiate a learned DNN claim: reported comparison figures are artifact results and the live prediction code uses explicit threshold rules. The next development milestone should be to implement and test the actual trained model path, validate it on independent representative data, and then repeat the complete gateway-enabled integration test.

REFERENCES

14. Project source repository: README.md; server/src/services/slidingWindowService.js; server/src/services/mlClientService.js; server/src/services/incidentService.js; ml-service/app/features.py; ml-service/app/model.py; ml-service/app/main.py; wokwi/sketch.ino and wokwi/diagram.json. Inspected for this report, 28 October 2026.
15. Wokwi ESP32 Wi-Fi guide. https://docs.wokwi.com/guides/esp32-wifi (accessed for gateway requirements as recorded in project documentation).
16. Wokwi simulation project: IoT Vehicle Accident Detection - Connected Wokwi Demo. https://wokwi.com/projects/479129585204028417
17. Eclipse Mosquitto documentation. https://mosquitto.org/documentation/
18. FastAPI documentation. https://fastapi.tiangolo.com/
19. Socket.IO documentation. https://socket.io/docs/v4/
20. TensorFlow documentation. https://www.tensorflow.org/guide
21. Saved evaluation artifacts: ml-service/reports/model_comparison.json; domain_shift_results.json; moment_evaluation.json; and ml-service/data/simulator_dataset.json.

APPENDIX A. RUN AND DEMONSTRATION PROCEDURE

A.1 Start the local stack

From the repository root, follow the README quick-start. The required local services are Mosquitto (1883), MongoDB (27017), backend (5000), ML service (8000), simulator control API (5055) and dashboard (5173). Confirm each is healthy before testing the device path.

A.2 Run the Wokwi circuit

22. Open the saved Wokwi project: https://wokwi.com/projects/479129585204028417. The sketch, diagram and required ArduinoJson/PubSubClient libraries are in the repository's wokwi folder.
23. For local MQTT integration, install/start the official Wokwi IoT Gateway and enable Wokwi Private IoT Gateway in the editor. Without this paid-plan feature, local broker connectivity is unavailable.
24. Start simulation and watch the serial output for READY, TIME_SYNCHRONIZED and MQTT_CONNECTED. An MQTT connection failure means the hardware simulation may still run locally, but data will not reach backend services.
25. Open the dashboard at http://localhost:5173, sign in, and select Fleet Monitor.
26. Change the MPU6050 accelX control to 5 to generate a simulated impact; test the CANCEL button within the configured countdown. Do not interpret the fixed GPS coordinates as live location.
27. Return accelX to 0 and restart the simulation to clear latched device alert state.

A.3 Expected verification

Layer	Expected evidence
Wokwi firmware	READY, TIME_SYNCHRONIZED, MQTT_CONNECTED when the gateway/broker path is configured.
MQTT	Device status and telemetry on vehicle/ESP32-WOKWI-001/*.
Backend	Device online state, rolling samples and candidate incident lifecycle.
ML service	POST /predict response; note current threshold-rule implementation qualification.
Dashboard	Fleet sample updates and pending/confirmed/cancelled incident events.

APPENDIX B. MESSAGE AND TEST RECORD

B.1 Illustrative telemetry shape

```
{
  "deviceId": "ESP32-WOKWI-001", "ts": 1793210000000,
  "seq": 25, "sampleRateHz": 50,
  "accel": {"x": 0.0, "y": 0.0, "z": 1.0},
  "gyro": {"x": 0.0, "y": 0.0, "z": 0.0},
  "gps": {"lat": 10.9982, "lng": 76.9664, "speed": 0,
          "heading": 0, "satellites": 0, "hdop": 0, "fix": 0},
  "buzzer": "OFF"
}
```

B.2 Evaluation source files

Artifact	What it records
model_comparison.json	Saved DNN-labelled, autoencoder and 3× noise metrics and confusion matrices.
domain_shift_results.json	Saved comparison on 3,000 domain-shift samples.
moment_evaluation.json	MOMENT NOT_EVALUATED status and PyTorch import failure.
simulator_dataset.json	Simulator/export dataset used by the local ML pipeline.

End of report. Before formal submission, fill in institution, department, project guide and signatures; confirm that the submitted repository revision and evaluation artifacts match the final demonstration.